

NAME: AKSHAYA A

REG. NO: 192421222

COURSE: CSA1402 COMPILER DESIGN

CO2 - AT3 - SCENARIO BASED TASK

1. scenario: Parsing table conflict in LL(1)

Ans: The LL(1) parser constructs its parse table using the FIRST and FOLLOW sets of the grammar. The FIRST set contains the terminals that can appear at the beginning of a production, while the FOLLOW set contains the terminal that can appear immediately after a non terminal. These set help the parser decide which production rule to apply.

Decision:

The grammar should be modified by eliminating left recursion, applying left factoring and ensuring that the FIRST and FOLLOW sets do not overlap.

This produces a conflict-free LL(1) grammar suitable for deterministic parsing.

2) Scenario: Incorrect Parse Tree Generation

The syntax analyzer constructs a parse tree by apply grammar rules to the input tokens. For the expression $id + id - id$, the parse tree determines the order in which operations are performed. If the grammar does not define operator associativity, the expression may be grouped incorrectly, leading to wrong evaluation results.

DECISION:

The grammar should enforce left associativity for the '+' and '-' operators. As a result, multiple parse trees can be generated for the same expression, creating ambiguity.

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow id$$

3) Scenario: Invalid Expression Handling

In the expression $a + * b$, the lexical analyzer correctly identify all tokens as valid identifiers and operators. However the syntax analyzer checks

whether these tokens follow the grammar rules. After the '+' operator, the grammar expects an operand, but another operator '*' appears instead. Therefore, the parser cannot generate a valid parse tree.

Decision:

The parser should implement error recovery techniques such as panic-mode recovery, phrase level recovery, or synchronization using FOLLOW sets. These methods help continue parsing and provide meaningful error messages.

4' scenario: Ambiguity in conditional statements

The grammar

$$S \rightarrow \text{if } E \text{ then } S$$
$$S \rightarrow \text{if } E \text{ then } S \text{ else } S$$
$$S \rightarrow a$$

is ambiguous because the parser cannot determine which if statement an else belongs to. This problem is called dangling else problem.

DECISION:

The grammar should be rewritten using matched and unmatched statements:

$$S \rightarrow M/U$$

$$M \rightarrow \text{if } E \text{ then } M \text{ else } M/a$$

$$U \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } M \text{ else } U$$

5. Scenario : operator precedence conflict

The grammar

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow \text{id}$$

is ambiguous because it does not define operator precedence or associativity. For the expression $\text{id} + \text{id} * \text{id}$ the parser may generate either $(\text{id} + \text{id}) * \text{id}$ or $\text{id} + (\text{id} * \text{id})$.

DECISION:

The grammar should be rewritten as:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow \text{id}$$